NOTE This is similar to the patterns you used in minimal APIs. But remember, minimal APIs use `IResult`, web API controllers, MVC controllers, and Razor Pages use `IActionResult`.

Listing 20.3 shows an example of where you must return an `IActionResult`. It shows another action on the same `FruitController` as before. This method exposes a way for clients to fetch a specific fruit by an `id`, which we'll assume for this example is an index into the list of `_fruit` you defined in the previous listing. Model binding is used to set the value of the `id` parameter from the request.

NOTE API controllers use the same model binding infrastructure as Razor Pages to bind action method parameters to the incoming request. Model binding and validation work the same way you saw in chapter 16: you can bind the request to simple primitives, as well as to complex C# objects. The only difference is that there isn't a `PageModel` with `[BindProperty]` properties; you can bind only to action method parameters.

Listing 20.3 A web API action returning `IActionResult` to handle error conditions

```
[HttpGet("fruit/{id}")]                          ❶
public ActionResult<string> View(int id)         ❷
{
    if (id >= 0 && id < _fruit.Count)            ❸
    {
        return _fruit[id];                       ❹
    }
    return NotFound();                           ❺
}
```

❶ Defines the route template for the action method

❷ The action method returns an ActionResult<string>, so it can return a string or an IActionResult.